

GERSL Management System Overview

Architecture, modules, and end-to-end workflows.

This document explains how GERSL's management platform connects every part of the organisation — from the moment a donor commits funding to the moment a beneficiary receives an asset and the report closes the loop back to the donor.

Use it as: an onboarding reference for new staff, a stakeholder briefing pack, and a working document that captures the operational rules baked into the system.

50+
pages

WEB ADMIN

**iOS +
Android**

MOBILE

14

MODULES

20+

ROLES

Table of Contents

1. Executive summary
2. The closed operational loop
3. Technical architecture
4. Module reference
5. WASH programme — full workflow
6. IGP programme — full workflow
7. Partner-wise finance integration
8. End-to-end case study (One Nation)
9. Mobile application
10. Roles and access control
11. Background automation (cron, hooks, email)
12. Database and schema overview
13. Audit, compliance, and reporting
14. Deployment and environment
15. What is intentionally parked
16. Appendix — API surface, env vars, glossary

1. Executive Summary

The GERSL Management System is a simple, powerful, and easy-to-use management system for a disaster relief NGO: orphan care, water/sanitation/hygiene, procurement, monitoring & evaluation, and a generic ERP — so every workflow matches the real world.

Three audiences, one system

- Field staff (mobile) — punch in, log visits, update programme stages in the field with GPS and photos, capture income follow-ups, submit fuel and expense claims.
- Office staff (web admin) — manage projects, partners, proposals, finance, HR, procurement, MEAL indicators, reports.
- Donors and partners — receive automated per-beneficiary PDFs and batch donor-bundle reports as work is delivered.

What was just built

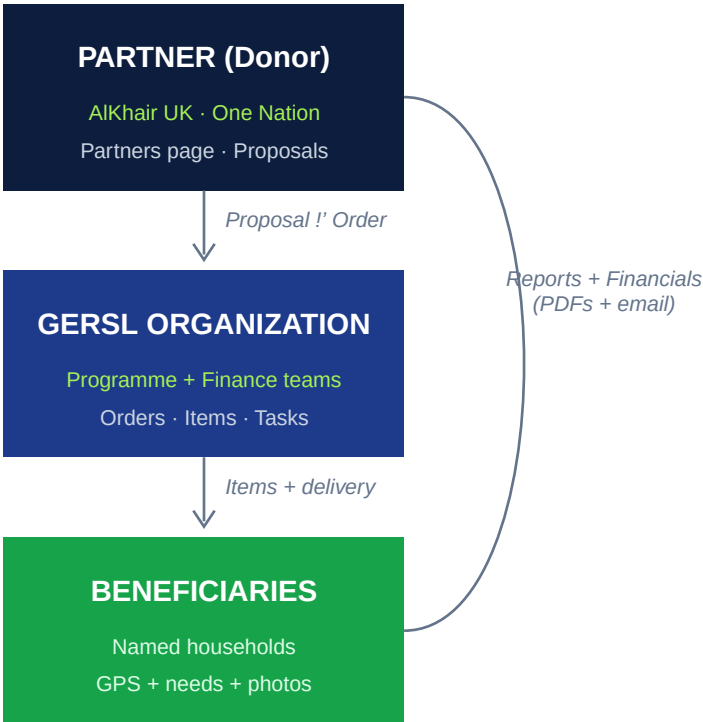
Module	Status	Key capabilities
WASH	Production	Donor order !' installation !' handover !' PDF
IGP	Production	Asset distribution + income follow-up tracking
Partner Financials	Production	Per-partner committed / received / spent / net
Beneficiary Map	Production	Leaflet map, layered WASH/IGP/Orphan/Households
Mobile programmes	Production	Stage update + map + follow-up + add-beneficiary
Auto-tasks	Production	Stage transition spawns next-step task
Deadline reminders	Production*	*requires CRON_SECRET on Vercel
Donor email	Production*	*requires SMTP env vars on Vercel

RELIABILITY POSTURE

The system follows a "best-effort outbound, atomic inbound" model. Email sends and PDF generation never block business operations — if SMTP is unconfigured, work proceeds and emails simply do not fire. Financial mutations (cash transactions, invoice reconciliation) are wrapped in transactions and audited.

2. The Closed Oper

GERSL's work is fundamentally a closed loop system. From the partner to the beneficiaries, reports return to the partner as the endpoint, every automated job in the system.



Money flows one way; work flows the other; — orders, invoices, items, tasks, PDFs, email

3. Technical Architecture

Layers

Layer	Stack	Hosted on
Web admin	React 19 + Vite 5 + Tailwind v3	Vercel
Server / API	Express 4 + Sequelize 6 (Node 20)	Vercel serverless
Database	PostgreSQL 15+	Supabase
Mobile	Flutter 3.x + Riverpod + Dio + go_router	Codemagic CI
File storage	Supabase Storage (uploaded photos, receipts)	Supabase
Email transport	SMTP via nodemailer	Bring-your-own SMTP host
Maps	OpenStreetMap tiles (web: react-leaflet · mobile: flutter_map)	OSM (free)

Auth model

- JWT access token (15 min) + refresh token (7 days). HttpOnly cookies primary; Bearer header fallback for the mobile app.
- Role-based permissions plus row-level scoping on programme data — field staff see only items they supervise; senior roles see all.
- Login + register + refresh are rate-limited (10 requests / 15 min) separately from the rest of the API to defend against brute force.
- CRON_SECRET protects scheduled-job HTTP triggers with constant-time comparison so the secret can't be probed character-by-character.

Code organisation

- server/src/ — controllers, routes, models, services, migrations
- src/ — web admin pages, components, services
- mobile/lib/ — Flutter features, repositories, services
- docs/ — generated documentation (this file)

4. Module Reference

Each module is a self-contained part of the system, often including (where relevant) mobile screens. Modules manage data for partners, beneficiaries, or projects across the system.

Dashboard /admin/dashboard
Role-tailored landing page with KPI tiles, charts, and quick links.

Orphan Care /admin/orphans
Orphan registry with dossier, sponsor link, and monitoring generated reports.

WASH /admin/wash
Donor-funded water/sanitation/hygiene interventions including filters, tanks, latrines.

IGP /admin/igp
Income-generating asset distribution. Sees and manages grants. Captures income baseline and follow-up.

Beneficiaries /admin/beneficiaries
Shared registry across programmes. NICs and photos across WASH, IGP, and Orphan Care.

Beneficiary Map /admin/map
Leaflet map with toggleable layers (WASH, IGP, Orphan Care).

Partners /admin/partners
Donor CRM with contributions log, commitments, and financials (committed/received/spent/net).

Proposals /admin/proposals
Funding proposals with approval workflow and conversion to Project or Order.

Projects /admin/projects
Project umbrella with budget vs actual, timeline, and reports.

Operations /admin/operations/*
Activities, Tasks (Kanban), My Tasks, Field queue, Compliance.

Finance /admin/finance
Invoices, bills, expenses, journal entries, and budget reports.

	Cash module /admin/finance/cash/* Locker / CashBook / PettyCash accounts
	HR /admin/hr Staff register, attendance, onboarding, ap register.
	Procurement /admin/procurement/* Inbox queue, RFQ workspace, PO mana
	MEAL /admin/meal Indicators, evaluations, learning events, c
	Campaigns /admin/campaigns Public fundraising campaigns with donati
	Reports /admin/reports Multi-format export (PDF, Excel, CSV), d
	Settings /admin/settings Users, roles, permissions, system setting

5. WASH Programm

WASH (Water, Sanitation, Hygiene) hand...
named beneficiary with GPS evidence. E...
village, a hybrid well, water filters, latrines...

Data model

Entity	Purpose
wash_orders	A batch from a donor. One Nation's 50 hand pumps for April = one wash_order.
wash_items	One installation. Linked to beneficiary, contractor, supervisor, GPS, stage.
wash_stage_updates	Append-only timeline of stage transitions with photos, GPS, notes.
partners	The donor (FK from order.donor_id).
projects	Umbrella that aggregates multiple orders (e.g. annual programme).
proposals	Upstream funding commitment (FK from order.proposal_id).
invoices	Downstream billing record (FK from order.invoice_id).
beneficiaries	Recipient registry (FK from item.beneficiary_id).
vendors	Contractor pool (FK from item.assigned_contractor_id).

Stage workflow

- Ordered** — Items registered against the order. No work yet.
- Surveyed** — Field officer visits the site, captures GPS + photos. Materials PR auto-task created.
- Materials** — Procurement raises a PO, materials delivered to site.
- Construction** — Contractor builds the installation. Progress photos captured.
- Testing** — MEAL Officer tests water quality / pump function.
- HandedOver** — Beneficiary takes ownership. Per-item PDF auto-generated.
- Reported** — Donor bundle PDF generated and emailed.

What happens at each transition

- Photos + GPS + notes recorded in wash_stage_updates (append-only).
- Item stage column updated to new value.
- Auto-task created for the next operational step (idempotent).
- Audit log entry written (who, when, what changed).
- At HandedOver: per-item donor PDF available immediately.

6. IGP Programme

IGP (Income Generating Programmes) h order is usually a single unit type, IGP or bicycles in one order). The asset mix is s

Stage workflow

- 1. Ordered** — Beneficiaries registered. No procurement yet.
- 2. Surveyed** — Field officer confirms eligibility, captures household details.
- 3. Procured** — Asset purchased / produced. Bill recorded against the partner.
- 4. Training** — Optional — for assets requiring skills (e.g. sewing machine course).
- 5. Delivered** — Asset handed to beneficiary. Photo + signature + GPS captured.
- 6. FollowUp** — Income follow-up visit 30–90 days later captures monthly income uplift.
- 7. Reported** — Donor bundle PDF generated with income tracking section.

Income tracking — the IGP di

IGP captures two figures per beneficiary: follow-up visit (typically 30–90 days after on the donor report so impact is visible p

FIELD WORKFLOW

Mobile users see "Record income follow-up" on the item detail screen after stage = Delivered. The bottom sheet captures the new income figure, notes, GPS, and optional photo. Stage automatically flips to FollowUp.

7. Partner-wise Financials

Until recently, expenses and bills could only be matched by manually matching the donor name. This is now fixed and is populated automatically through database queries.

Data flow

Tile (UI)	Computed from
Committed	$\text{SUM}(\text{wash_orders.total_budget}) + \text{SUM}(\text{igp_orders.total_budget})$ where donor_id = partner
Received	$\text{SUM}(\text{invoices.paid_amount}) + \text{SUM}(\text{partner_contributions.amount})$ where partner_id = partner
Spent	$\text{SUM}(\text{wash_items.actual_cost}) + \text{SUM}(\text{igp_items.actual_cost}) + \text{SUM}(\text{expenses.amount}) + \text{SUM}(\text{bills.paid_amount})$ where partner_id = partner
Net position	Received " Spent
Outstanding	$\text{SUM}(\text{invoices.total_amount}) - \text{invoices.paid_amount}$

Auto-population hooks

- Expense.beforeCreate — if expense has projectId but no partnerId, looks up the project and copies its partner_id automatically.
- Bill.beforeCreate — same pattern; bills against a contractor on a partner-linked project roll up to that partner.
- Invoice.afterUpdate — when paid_amount changes, finds any linked WASH/IGP orders and updates their payment_status. This is what unlocks "OnFundsReceived" orders the moment donor funds arrive.

Access control

The /api/partners/:id/financials endpoint is restricted to senior roles (Admin, CEO, Finance Manager, Finance Officer, Accountant, Fundraising Manager, Director Programmes, Programme Manager). General PARTNERS_VIEW permission does not unlock the financial breakdown — that exposes contractor bills and cash flows that should not be visible to general staff.

8. End-to-End Case

One Nation March — 50 IGP a

Day 1	Programme Officer creates Proposal in /admin/proposals: 15 sewing machines + 15 canoes + 20 bicycles. proposalType = IGP, proposalLineItems = JSON array. Status = Draft.
Day 2	Proposal submitted !' CEO + Finance + Programme Manager approve. Status flips through Submitted !' Under Review !' Approved. PDF auto-generated.
Day 3	PDF emailed to One Nation (via PartnerCommunication log). Status = Submitted to Donor.
Day 7	One Nation approves. Donor decision logged. Status = Donor Approved. "Convert to Order" button appears.
Day 7	One click converts proposal to IGP order IO-2026-001. assetMix copied from proposalLineItems. 50 empty items pre-stubbed because namedBeneficiaries = true. proposal.convertedOrderId set bidirectionally.
Day 7	"Generate Invoice" button creates a proforma Invoice INV-2026-IGP-00001 (because donor.paymentTerm = AdvanceProforma). Auto-emailed to One Nation.
Day 12	One Nation transfers funds. Finance Officer records the BankTransaction and reconciles it against the invoice. invoice.paid_amount = full amount. Invoice.afterUpdate hook fires !' IO-2026-001.paymentStatus auto-flips to Paid !' all 50 auto-tasks for "Conduct site survey" assigned.
Day 15–25	Programme Officer bulk-imports 50 beneficiary names via CSV. Each row becomes a Beneficiary + IgplItem. GPS coordinates validated against Sri Lanka bounds.
Day 26–50	Field officers update items on mobile. Surveyed !' Procured !' Training !' Delivered. Each transition captures photo + GPS. Per-item PDF auto-generated at Delivered.
Day 60+	Mobile field officers visit beneficiaries 30 days post-delivery. Tap "Record income follow-up". incomeFollowup figure captured, uplift computed against baseline.
Day 65	Programme Manager clicks "Email donor" on the order. Server generates donor-bundle PDF (cover sheet + 50 item pages with photos, GPS, income tracking, signatures). PDF attached to email to One Nation contact.
Day 66+	One Nation receives the bundle. Reviews impact. Opens conversation for next quarter. Partner.Financials panel shows Committed/Received/Spent/Net = closed loop.

9. Mobile Application

The mobile app is what field staff use day to day. It supports the operational flows that need to be done in the field.

Feature matrix

Feature	Mobile	Web admin
Punch in/out (face detection, GPS, selfie)	Yes	View only
Attendance summary	Yes	Yes
My Tasks	Yes	Yes
Visits (general field)	Yes	Yes
Movements (vehicle trips)	Yes	Yes
Fuel claims	Submit	Approve
Leave / Expense / Advance request	Submit	Approve
Payslip view + download PDF	Yes	Generate
Vehicle / accommodation requests	Submit	Approve
Compliance incident report	Submit	Manage
Announcements	Read	Manage
Notifications	Yes	Yes
My Programmes (WASH + IGP items)	Yes	Yes
Item stage transitions + photo + GPS	Yes	Yes
Item detail with stage timeline	Yes	Yes
Per-item PDF view / share	Yes	Yes
Order-level view + sibling items	Yes	Yes
Add beneficiary during field survey	Yes	Bulk import
IGP income follow-up capture	Yes	Yes
Beneficiary map (Leaflet / OSM)	Yes	Yes
Funds-gate warning on unpaid orders	Yes	Yes

Build & distribution

- Codemagic CI on push to main; produces signed .apk and .ipa
- iOS uploads to TestFlight; Android to Play Console internal track
- flutter_map + latlong2 dependencies handle the in-app map

- Sentry wired (DSN optional — silent without it)

10. Roles and Access

GERSL uses both role-based permissions and row-level scoping. What a user can see; row-level scoping decides what a user can do.

Senior roles (full org-wide visibility)

Admin · CEO · BOD · Programme Manager · Finance Officer · Accountant · MEAL Officer

Field-level roles (scoped to the field)

Anyone outside the senior set sees only their own field. Field-level roles include Field Officer, Coordinator, Driver, and any customer-facing roles.

HOW TO ADD A NEW SENIOR ROLE

Edit server/src/utils/programmeAccess.js — FULL_VIEW_ROLES is the single list to update. A code change is required (one-line addition). No database migration needed.

Authentication layers

- HttpOnly access cookie (primary, 15-min expiry)
- Bearer header (fallback, for mobile + cross-origin)
- Refresh token (7-day, stored in DB; rotation not yet enabled)
- CRON_SECRET via x-cron-secret header (timing-safe comparison)
- ?token=... query parameter allow-listed only for PDF endpoints

11. Background Au

Database hooks

Hook	What it does
Expense.beforeCreate	Inherits partner_id from project if not set.
Bill.beforeCreate	Same — inherits partner_id from project.
Invoice.afterUpdate	When paid_amount changes, syncs payment_status onto linked WASH/IGP orders.
Stage transition (controller)	Writes stage_updates row + creates next-step Task (idempotent).
withAuditLog (every audited model)	Records create/update/delete to audit_logs.

Scheduled jobs (cron)

- Daily 01:30 — movement clusterer aggregates location_points into movement_segments for fuel-claim distance calculations.
- Daily 08:00 — programme deadline reminder digest. Finds at-risk WASH/IGP items and emails each supervisor a personalised list of items overdue, due tomorrow, or due this week.

VERCEL CRON SETUP

In-process cron only fires on long-running servers, not on Vercel serverless. For production, configure a vercel.json cron entry pointing at POST /api/cron/programme-deadline-reminders with header x-cron-secret set to your CRON_SECRET env var.

Email events

- Leave / expense / salary advance submitted !' notify approvers
- Decision (approve / reject) !' notify the requester
- Donation recorded !' automated donor receipt PDF
- Programme order completion !' donor bundle PDF on request
- Deadline reminder digest !' daily 08:00 (when cron wired)

12. Database and S

PostgreSQL on Supabase. Schema man...
 provides the ORM layer on top with abou...
 to re-run.

Core entity groups

Group	Tables
Identity	users, roles, permissions, role_permissions
Org	departments, positions, staff, staff_documents
Programmes	orphans, wash_orders, wash_items, wash_stage_updates, igp_orders, igp_items, igp_stage_updates, beneficiaries, beneficiary_supports
Partners	partners, partner_contributions, partner_communications
Finance	invoices, bills, expenses, journal_entries, accounts, bank_accounts, bank_transactions, cash_accounts, cash_transactions, cash_count_sessions, petty_cash_replenishments
Projects	projects, project_team_members, project_indicators, evaluations, learning_events, complaints
Proposals	proposals
Procurement	purchase_requisitions, rfqs, rfq_vendors, quotations, quotation_lines, bid_analyses, purchase_orders, po_lines, goods_receipt_notes, grn_lines, three_way_matches, vendors, vendor_calls, vendor_submissions
HR	leave_requests, leave_balances, salary_advances, payrolls, employment_agreements, contract_renewals, terminations, resignations, attendances, attendance_punches, attendance_corrections
Mobile / field	location_points, movement_logs, movement_segments, fuel_claims, fuel_claim_passengers, fuel_rates, vehicles, visits, vehicle_requests, accommodation_requests
Audit / logs	audit_logs, notifications, announcements

Key foreign-key links (closed loop)

- partners !• wash_orders.donor_id, igp_orders.donor_id
- projects !• wash_orders.project_id, expenses.project_id, bills.project_id
- proposals !• wash_orders.proposal_id, igp_orders.proposal_id (bidirectional)
- invoices !• wash_orders.invoice_id, igp_orders.invoice_id
- beneficiaries !• wash_items.beneficiary_id, igp_items.beneficiary_id, orphans (parallel registry)
- vendors !• wash_items.assigned_contractor_id, bills.vendor (string)
- users !• *.created_by, *.assigned_supervisor_id, *.updated_by
- partners !• projects.partner_id, expenses.partner_id, bills.partner_id, invoices.partner_id

13. Audit, Compliance

Audit log coverage

The audit_logs table captures every creation and update of data in the system to ensure donor accountability. Every entry records the user who made the change and the date and time of the change. Currently audited:

- Users, Roles, Role-Permissions
- Projects, Proposals, Approvals
- Beneficiaries, Vendors, Employment Agreements
- Bills, Payments, Donations, Journal Entries
- Cash Accounts, Cash Transactions, Cash Count Sessions, Petty Cash Replenishments
- Purchase Requisitions, RFQs, Quotations, Bid Analyses, POs, GRNs, Three-Way Matches
- WASH Orders, WASH Items, IGP Orders, IGP Items (newly added this release)
- Attendance, Punches, Corrections, Fuel Rates, Fuel Claims, Movements

Generated documents

Document	Where	Generated by
Cash voucher PDF (per transaction)	/admin/finance/cash	On-demand
Cash book PDF (per account, range)	/admin/finance/cash	On-demand
Payslip PDF	/admin/hr/payroll	On payroll run
Fuel claim PDF	/admin/operations/fuel-claims	On approval
Invoice PDF	/admin/finance	On invoice generation
Donation receipt	(email)	On payment status = Completed
WASH/IGP per-item donor PDF	Item detail	On-demand or stage = Reported
WASH/IGP donor bundle PDF	Order detail	On-demand or end of cycle
Proposal PDF	/admin/proposals	On submission
Project completion report	/admin/projects/:id	On-demand

14. Deployment and

Production stack

Service	Provider	Purpose
Web + API	Vercel	Auto-deploy on push to main. Serverless functions for /api routes.
Database	Supabase	PostgreSQL with automatic backups. SQL editor for migrations.
Mobile CI	Codemagic	Manual trigger; auto-signs and uploads to TestFlight + Play.
Email	SMTP	Bring-your-own (SendGrid, Mailgun, etc.). Dormant until configured.
Storage	Supabase Storage	Uploaded photos, receipts, fuel claim attachments.

Required environment variables (Vercel)

Variable	Required for	Notes
DATABASE_URL	Always	Supabase Postgres connection string
JWT_SECRET	Always	Secret used to sign access + refresh tokens
JWT_REFRESH_SECRET	Always	Separate from JWT_SECRET
APP_BASE_URL	Email links	Your Vercel URL, no trailing slash
SMTP_HOST	Email features	Without this, emails short-circuit silently
SMTP_PORT	Email features	Default 587
SMTP_USER	Email features	SMTP auth username
SMTP_PASS	Email features	SMTP auth password
EMAIL_FROM	Email features	e.g. "GERSL <noreply@gersl.org>"
CRON_SECRET	Vercel Cron	Random "e32-char string; matches x-cron-secret header
SENTRY_DSN	Error tracking	Optional. App runs without it.

SQL migration history

- add_visits_orphan_columns.sql — orphan visit linkage
- add_hot_column_indexes.sql — performance indexes (corrected snake_case)
- create_wash_igp_modules.sql — WASH/IGP tables + partner/proposal extensions
- extend_staff_columns.sql — HR profile columns for Edit Staff form

- add_partner_id_to_finance_tables.sql — partner FK + backfill
- add_programme_composite_indexes.sql — cron query optimisation

MIGRATION POLICY

All migrations are idempotent (CREATE ... IF NOT EXISTS, ADD COLUMN IF NOT EXISTS). Safe to re-run. To apply, paste contents into Supabase SQL Editor and run. No migration framework currently tracks applied state.

15. What is Intention

These features have been considered or need stakeholder decisions, additional bu

MOBILE PUSH NOTIFICATIONS (P

The mobile app currently relies on email would add proper push. Estimated effort:

PARTNER SELF-SERVICE PORTA

Donors currently receive PDF reports by invoices, and reports would replace most tokenised share links per partner (2 hour

BANK !” CASH RECONCILIATION

Currently manual: finance officer reconcil reconciliation screen with matching algor

AUTO JOURNAL-ENTRY CREATIO

Every cash transaction would ideally gen Currently those are entered manually. Mu modules.

REFRESH TOKEN ROTATION

Refresh tokens are reused for the full 7-d Few-hour change but needs careful rollo

BILL.LINEITEMS JSON !’ JSONB

Cosmetic indexing improvement. Will spe table; defer until needed.

16. Appendix

A. REST API surface (high level)

Prefix	Module
/api/auth/*	Login, register, refresh, logout, password reset
/api/users/*	User management, profile, current user
/api/orphans/*	Orphan registry, visits, reports
/api/wash/*	WASH orders, items, stage updates, summary, my items
/api/igp/*	IGP orders, items, stage updates, follow-up, summary
/api/map/pins	Unified map pins (WASH + IGP + Orphan + Households)
/api/partners/*	Partner CRM + financial rollout
/api/proposals/*	Proposals + convert-to-project + convert-to-order
/api/projects/*	Projects + budget vs actual + indicators
/api/finance/*	Invoices, bills, expenses, journal entries, accounts
/api/cash/*	Cash accounts, transactions, vouchers, replenishments
/api/hr/*	Staff register, payroll, leave, expenses, advances
/api/procurement/*	PRs, RFQs, POs, GRNs, vendors, three-way match
/api/me/*	Self-service for mobile (visits, leaves, expenses)
/api/cron/*	Scheduled-job triggers (auth via x-cron-secret)
/api/audit-logs	Audit log access (Admin/CEO/BOD only)

B. Glossary

WASH — Water, Sanitation, Hygiene — programme delivering installations to beneficiaries.

IGP — Income Generating Programmes — livelihood asset distribution.

MEAL — Monitoring, Evaluation, Accountability, Learning — programme quality function.

Partner — Donor organisation funding programmes (e.g. AlKhair UK, One Nation).

Beneficiary — Named individual receiving support — household head or guardian.

Order — A batch of work from a donor (e.g. 50 hand pumps for April).

Item — One installation/asset within an order — one beneficiary, one GPS point.

Stage — Operational status of an item (Ordered !' Surveyed !' ... !' Reported).

Stage update — Append-only timeline entry with photos, GPS, notes per transition.

Funds-gate — Rule that blocks (or warns) on work starting before donor pays.

Donor bundle — PDF report combining a cover sheet with one page per item.

Voucher — Cash transaction printout (A5 PDF) for audit attachment.

Three-way match — Procurement check: PO + GRN + vendor invoice must agree.

C. Regenerating this document

This PDF is generated from scripts/generate_app_overview.js on the server package. To refresh after major feature

```
cd <repo-root>
node scripts/generate_app_overview.js
# !' docs/GERSL-Management-Overview.pdf
```

Add new sections by editing the script. Each h1/h2/p/bullet/table helper handles pagination automatically, so you can append content top-to-bottom without managing page breaks manually.

